

Locality Driven State Management for In-Memory Stream Processing

Ben Trovato*
G.K.M. Tobin*
trovato@corporation.com
webmaster@marysville-ohio.com
Institute for Clarity in Documentation
Dublin, Ohio, USA

Lars Thørväld
The Thørväld Group
Hekla, Iceland
larst@affiliation.org

Valerie Béranger
Inria Paris-Rocquencourt
Rocquencourt, France

Aparna Patel
Rajiv Gandhi University
Doimukh, Arunachal Pradesh, India

Huifen Chan
Tsinghua University
Haidian Qu, Beijing Shi, China

Charles Palmer
Palmer Research Laboratories
San Antonio, Texas, USA
cpalmer@prl.com

John Smith
The Thørväld Group
Hekla, Iceland
jsmith@affiliation.org

Julius P. Kumquat
The Kumquat Consortium
New York, USA
jpkumquat@consortium.net

Abstract

In-memory state-intensive stream processing is often limited less by operator computation than by the physical organization of state. Under frequent updates, window maintenance, and timer firing, fragmented heap layouts and long indirection chains turn routine state accesses into cache-miss-heavy, memory-bound execution, inflating tail latency and runtime jitter in latency-sensitive deployments, especially when memory budgets are tight. Existing in-memory backends largely prioritize semantic generality, JVM integration, and checkpoint compatibility, leaving locality and working-set control underexploited. We therefore design a locality-driven Flink state backend that combines compact metadata-guided probing, namespace-aware hierarchical partitioning, and a lightweight native storage engine. The implementation uses a thin Java compatibility layer over native hot-path storage. The evaluation combines isolated microbenchmarks and end-to-end streaming workloads, comparing against standard in-memory baselines with throughput, tail latency, cache-miss intensity, memory-bound stall breakdown, GC behavior, and memory footprint as primary metrics, including memory-constrained settings. The broader conclusion is that improving in-memory stream processing requires rethinking the physical organization of state, not merely accelerating individual state operations, while preserving Flink semantics and snapshot compatibility.

*Both authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

CCS Concepts

• **Do Not Use This Code** → **Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

stream processing, state management, cache locality, hash table

ACM Reference Format:

Ben Trovato, G.K.M. Tobin, Lars Thørväld, Valerie Béranger, Aparna Patel, Huifen Chan, Charles Palmer, John Smith, and Julius P. Kumquat. 2018. Locality Driven State Management for In-Memory Stream Processing. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 10 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Stateful stream processing systems underpin real-time analytics, fraud detection, market surveillance, industrial monitoring, and other latency-sensitive applications in which both correctness and runtime stability matter. Modern engines provide mature semantics for time, windows, fault tolerance, and recovery, but these guarantees rely on reading and updating state for nearly every incoming record [1–5]. As workloads become more state-intensive, the dominant cost increasingly shifts away from operator computation and toward the backend that serves keyed state, window state, and timer-related accesses.

For in-memory backends, this shift exposes a memory-system bottleneck. Frequent per-record updates, window maintenance, and timer firing create irregular accesses over large keyed state, so even memory-resident state can become strongly memory-bound. The symptoms are high cache-miss rates, long dependent access chains, inflated tail latency, and runtime jitter. These effects matter in practice because many deployments operate under tight memory

budgets, and backend inefficiency translates directly into lower throughput, worse p99 latency, or higher infrastructure cost.

The difficulty is that existing in-memory backends are usually optimized first for generality, semantic correctness, JVM integration, and checkpoint or recovery support, rather than for cache locality. As a result, state is often represented as pointer-heavy object graphs with separated metadata and payload, while routing a single logical access may traverse several layers of generic containers before reaching useful data. Namespace structure is also underexploited: hot windows or timer-related sub-state can interfere with unrelated namespaces inside larger tables, expanding the working set and worsening cache thrashing. In other words, the backend problem is not merely hash computation; it is the physical organization of state in memory.

This paper takes that view as its starting point. We present a locality-driven state backend for Flink built around three complementary ideas. First, we use compact metadata-guided probing: control-byte fingerprints and word-parallel matching filter candidate slots before touching payload, while a slot-centric layout keeps each key association adjacent to its value. Second, we organize state hierarchically as state $\rightarrow$ key group $\rightarrow$ namespace $\rightarrow$ sub-table so that hot working sets become smaller and more stable under namespace-heavy workloads. Third, we move the hot-path storage engine into lightweight native code, allowing tighter control over memory layout and reducing JVM heap-object pressure without changing the external state model.

The implementation uses a split architecture. Java remains the compatibility shell responsible for Flink API integration, state registration, serializer and type analysis, metadata tracking, and snapshot orchestration, while native code holds the off-heap state engine and serves the critical read and write path. Within the native engine, each registered state is partitioned by key group and namespace and stored in compact sub-tables. This architecture is designed to preserve Flink’s keyed-state semantics and snapshot or restore compatibility while changing only the physical realization of in-memory state.

The goal of this paper is therefore not to claim that a standalone hash table is faster in isolation. The goal is to show that cache-friendly layout, namespace-aware organization, and reduced JVM-heap dependence together form a practical backend design for state-intensive stream processing. To test that claim, our evaluation combines isolated microbenchmarks based on flink-state-benchmark with end-to-end streaming workloads from benchset. We focus on throughput, average and tail latency, cache misses, memory-bound stall breakdown, GC disturbance, and memory footprint, with particular attention to memory-constrained settings where heap pressure is expected to matter most.

Our contributions are:

- We argue that backend performance for in-memory, state-intensive stream processing should be understood primarily as a memory-layout and memory-management problem.
- We present a locality-driven Flink state backend that combines compact metadata-guided probing, slot-centric payload placement, and namespace-aware hierarchical partitioning.

- We implement the design as a Java compatibility shell plus a lightweight native state engine, preserving Flink state semantics and snapshot compatibility while shortening the hot path and reducing heap pressure.
- We define an evaluation methodology that measures not only throughput but also tail latency, cache misses, memory-bound stalls, GC behavior, and memory footprint under both regular and memory-constrained workloads.

2 Background and Motivation

2.1 Problem Statement

Modern stream processing applications increasingly rely on large, mutable state for correctness and expressiveness. However, the dominant cost of stateful execution is often not arithmetic, but memory stalls triggered by state access: lookups and updates traverse scattered allocations, repeatedly missing in caches and amplifying tail latency. This behavior is especially pronounced when keys are high-cardinality and access patterns approximate random probes across large memory regions.

We formalize the problem as locality-aware state management for keyed stream processing. Let $\mathcal{K}$ be the key domain, $\mathcal{N}$ the namespace domain (e.g., windows or operator sub-state), and $\mathcal{S}$ the set of logical state variables (e.g., value state, map state). A stream execution induces a sequence of state operations $\mathcal{O} = \langle o_1, o_2, \dots, o_m \rangle$, where each operation is of the form $o_i = (\text{op}_i, k_i, n_i, s_i, x_i)$, with $k_i \in \mathcal{K}$, $n_i \in \mathcal{N}$, $s_i \in \mathcal{S}$, and x_i an optional payload (e.g., a value to write). The state backend implements the semantics of these operations (e.g., lookup, update, delete) and must preserve correctness and recoverability.

We view a backend design as choosing a physical organization Θ (layout + routing) that maps each logical access to a concrete sequence of memory references (metadata reads, key validation reads, and value reads/writes). Given a workload distribution over operations, our goal is to reduce cache-miss-driven stalls so as to minimize latency (and thereby maximize throughput), while preserving the semantics of state operations:

$$\min_{\Theta} \mathbb{E}_{o \sim \mathcal{O}} [T(o; \Theta)] \quad (1)$$

$$\text{s.t. } \forall o \in \mathcal{O} : \text{Sem}(o; \Theta) = \text{Sem}(o; \Theta_{\text{baseline}}). \quad (2)$$

Here $T(o; \Theta)$ is the per-operation latency contribution of state access on the critical path. For a fixed operator pipeline and input rate, reducing $\mathbb{E}[T]$ increases sustainable throughput; conversely, throughput improvements can be viewed as a consequence of lowering the stall component of T . To connect the objective to cache behavior, we use a locality-centric model that emphasizes memory stalls due to cache misses:

$$T(o; \Theta) \approx T_{\text{cpu}}(o; \Theta) + \alpha \text{Miss}_{\text{LLC}}(o; \Theta) + \beta \text{Miss}_{\text{DRAM}}(o; \Theta), \quad (3)$$

where Miss_{LLC} and $\text{Miss}_{\text{DRAM}}$ denote last-level cache and DRAM miss events attributable to state access, and α, β reflect their amortized stall costs. In memory-bound regimes, lowering miss intensity is therefore the most direct way to reduce latency and improve throughput.

Figure 1 sketches the baseline access path and the root causes of cache-miss amplification. In a typical in-memory backend, a single logical state operation consults metadata, follows multiple pointers

Placeholder. Replace this box with a schematic figure. Suggested content: (i) baseline access path (logical request → routing → lookup/update), (ii) fragmented metadata/payload layout, (iii) cache-miss amplification under window/timer-driven bursts, and (iv) where the redesign improves locality.

Figure 1: Baseline state access path in common in-memory backends and where cache misses are amplified under state-heavy streaming workloads.

Placeholder (Primary study). Cache miss rate of Flink Heap state backend. Replace with a plot (e.g., LLC misses per kilo-instruction or miss ratio) across workloads/state sizes.

Figure 2: Cache-miss intensity of Flink Heap state backend under state-heavy workloads (placeholder).

to locate a candidate entry, and then fetches key/value material from separately allocated objects. Because these memory locations are weakly correlated across consecutive records, hardware prefetching is ineffective and latency is dominated by last-level cache and DRAM misses.

The problem is exacerbated by two structural properties of common in-memory state organizations. First, state is frequently stored in fragmented layouts with per-entry object overhead, which separates metadata from payload and forces extra cache lines to be touched during a single access. Second, the access path is layered: routing through multiple abstraction levels introduces additional indirections and branches, making state operations sensitive to pointer chasing rather than data movement.

2.2 Motivation

Streaming workloads intensify these issues because they couple high update rates with time- and window-driven maintenance. Windowed operators repeatedly read-modify-write state for the same logical key over time, while timers introduce bursts of accesses that compete for cache capacity. Under sustained throughput, this mix increases cache churn and makes performance hinge on how efficiently the state layout serves irregular, fine-grained accesses.

To ground the motivation in measurements, we first characterize Flink’s default in-memory Heap state backend under representative state-heavy streaming workloads (update-heavy keyed operators with windows/timers). Using hardware performance counters, we observe that state access exhibits (i) high cache-miss intensity (especially at the last-level cache) and (ii) a large fraction of cycles spent in memory-bound execution, with substantial pressure attributable to both L3 and DRAM. Figures 2 and 3 summarize these observations. Together, these measurements indicate that, for such workloads, performance is limited less by compute and more by locality and data movement, motivating a backend design that directly targets cache behavior and memory stalls.

Placeholder (Primary study). Top-down breakdown showing high *Memory Bound*, with large *L3 Bound* and *DRAM Bound* components. Replace with a stacked-bar (or similar) across workloads.

Figure 3: Top-down analysis of execution stalls for Flink Heap state backend (placeholder).

These observations are consistent with a broad literature on stateful streaming systems, which has largely prioritized semantics, fault tolerance, and operational concerns such as recovery, out-of-order handling, and elasticity [1–3, 5, 11]. Other lines of work emphasize incremental or transaction-oriented processing for streaming analytics [9, 10, 12], addressing how to maintain and update derived state efficiently as inputs evolve. Across these systems, state remains a central abstraction, but the steady-state cost of fine-grained state access is often treated as an implementation detail behind a backend interface; in practice, many deployments still rely on general-purpose in-memory structures or embedded key-value stores, where object overhead and indirection can make performance highly sensitive to cache behavior [3]. This motivates a complementary focus: improving the cache locality and access-path length of the in-memory state representation itself, while keeping the established semantics and operational contract intact.

These observations motivate a design goal beyond a faster lookup primitive: state storage should be explicitly locality-aware. We seek compactness to reduce footprint and cache working-set size, short access paths to minimize indirections and branches, and layout choices that co-locate the information needed for probing and validation within a small number of cache lines. Finally, because stream processing systems depend on strict state semantics and operational tooling, these improvements must preserve existing correctness guarantees.

3 Challenges

C1. Random access amplified by fragmentation. State lookups and updates approximate random probes over a high-cardinality key space, so performance quickly becomes dominated by cache misses rather than by hash computation. Existing in-memory backends often represent state as a graph of heap objects (per-entry objects, boxed keys/values, and separate metadata), which fragments logically adjacent data. As a result, a single logical access touches multiple cache lines across distant addresses (routing metadata, bucket/entry metadata, key material, and the value), inflating last-level cache and DRAM misses when the working set exceeds cache capacity. The core difficulty is that locality must be improved without changing the observable semantics of state operations.

C2. Long access chains and dependent pointer chasing. Beyond layout, the state access path itself is often layered: routing through key-group selection, namespace maps, and generic container abstractions introduces multiple pointer dereferences and branches before any useful payload is reached. These dependent loads serialize (the next address is unknown until the previous load completes), which reduces memory-level parallelism and makes

hardware prefetching ineffective. Consequently, even when operator logic is simple, the end-to-end per-record latency is dominated by stall time on the critical path.

C3. Namespace interference and unstable working sets. Streaming state is naturally scoped by execution context (key group and namespace, e.g., windows/timers), and at any moment only a small subset of namespaces may be hot. However, generic storage frequently co-mingles many namespaces in large tables or treats namespaces as an afterthought, so unrelated working sets compete for the same cache capacity. Timer firing and window maintenance further create bursty access patterns that evict useful data and amplify capacity/conflict misses, increasing latency variance and hurting tail performance. The challenge is to exploit namespace structure to isolate working sets and reduce cache interference under mixed and bursty workloads.

These challenges motivate the mechanisms described in Section 4.

4 Design

This section presents a locality-driven state-store design built around three key ideas. Together they reduce cache-miss-driven stalls and dependent pointer chasing, directly targeting the objective in Eq. (2) and the miss-dominated cost model in Eq. (3). Each idea resolves one of the challenges (C1–C3) in Section 3.

4.1 Design overview

Our goal is to minimize the per-record state-access latency $T(o; \Theta)$ (Eq. 2) by lowering the cache-miss and memory-stall components that dominate memory-bound execution (Eq. 3). We do so with three design ideas:

- **Compact metadata-guided probing and layout** (addresses C1): keep probing metadata compact and cache-resident, and touch payload (keys/values) only for a small set of candidates; co-locate key association and value to reduce cache-line fetches on hits.
- **Lightweight routing on the hot path** (addresses C2): shorten the access chain from a logical state request to the physical slot by reducing abstraction layers, branches, and dependent pointer dereferences.
- **Namespace-partitioned state organization** (addresses C3): align physical partitioning with streaming execution context (key group and namespace) so that hot working sets stay small and interference across namespaces is reduced.

Figure 4 sketches the end-to-end pipeline. A record is routed to a key group and namespace, then served by a compact sub-table that first probes control metadata and only then validates candidate keys and reads/writes the value.

4.2 Compact metadata-guided probing and layout (C1)

This mechanism addresses C1 by reducing the number of cache lines touched per operation and by making the first probe stage mostly metadata-only. It directly targets the miss terms in Eq. (3) by (i) keeping control metadata compact and cache-resident, and (ii)

Placeholder. Replace this box with an end-to-end pipeline schematic. Suggested content: record $\rightarrow$ key-group routing $\rightarrow$ namespace routing $\rightarrow$ sub-table probe (control bytes $\rightarrow$ candidate lanes $\rightarrow$ key validation $\rightarrow$ value read/write). Annotate which steps are metadata-only vs. payload-touching.

Figure 4: End-to-end overview of the locality-aware state access pipeline.

Placeholder. Replace this box with a sub-table layout and probe schematic. Suggested content: (i) 8-slot group with control bytes (EMPTY/DELETED/fingerprint), (ii) payload region with slot-centric key/value adjacency, (iii) probe flow: load ctrl word $\rightarrow$ parallel match $\rightarrow$ validate candidate keys $\rightarrow$ read/write values, (iv) early-stop on EMPTY.

Figure 5: Sub-table layout and metadata-guided probing using control-byte fingerprints.

minimizing payload touches that would otherwise trigger LLC/DRAM misses.

We perform *metadata-first probing*: each sub-table is an open-addressed hash structure with a compact control region (one byte per slot) that stores a short fingerprint and encodes EMPTY/DELETED/FULL. Probing proceeds in 8-slot groups: a lookup loads a control-word for the group, performs word-parallel comparisons to identify lanes whose fingerprints match, and only then validates candidates against the key. This reduces branchy per-slot checks and enables early termination when an empty control byte is observed.

Figure 5 illustrates the sub-table layout and the two-stage probe that scans control metadata before touching keys and values.

To reduce cache-line fetches on hits, the payload region uses a slot-centric layout that keeps each key association adjacent to its value. When a fingerprint match is found, candidate validation reads key material from a compact representation when available; otherwise it dereferences non-inlined key material. In both cases, adjacency ensures that a hit triggers a small number of localized payload touches.

Because streaming workloads are update-heavy, the write path mirrors the read path: probe metadata, validate candidates, then update in place or claim an available slot. Deletions create tombstones to preserve probe invariants. Tombstones can lengthen probes under churn; we therefore treat tombstone ratio as a first-class signal and trigger rehashing when probe efficiency degrades.

In the notation of Eq. (3), metadata-first probing reduces Miss_{LLC} and $\text{Miss}_{\text{DRAM}}$ by avoiding unnecessary payload reads, and adjacency reduces the number of distinct cache lines fetched on hits. Maintenance (rehash/grow) preserves these benefits by preventing probe chains from growing due to tombstones or high load.

Algorithms 1–3 summarize the lookup, insert/update, and rebuild logic at a high level.

Algorithm 1 Metadata-guided probe (lookup path).

```

1: procedure PROBE( $T, h, k$ )
2:    $g \leftarrow \text{GROUPINDEX}(h)$   $\triangleright$  starting group from high hash bits
3:    $f \leftarrow \text{FINGERPRINT}(h)$   $\triangleright$  small hash fragment stored in
   control bytes
4:   loop
5:      $ctrl \leftarrow \text{LOADCTRLWORD}(T, g)$ 
6:      $mask \leftarrow \text{MATCHFINGERPRINT}(ctrl, f)$ 
7:     while  $mask \neq 0$  do
8:        $i \leftarrow \text{NEXTLANE}(mask)$ 
9:        $s \leftarrow \text{GROUPBASE}(g) + i$ 
10:       $a \leftarrow \text{SLOTKEYASSOC}(T, s)$ 
11:      if  $\text{KEYEQUALS}(a, k)$  then
12:        return  $s$ 
13:      end if
14:       $mask \leftarrow \text{CLEARLANE}(mask, i)$ 
15:    end while
16:    if  $\text{HASEMPTY}(ctrl)$  then
17:      return NOTFOUND
18:    end if
19:     $g \leftarrow \text{NEXTGROUP}(g)$ 
20:  end loop
21: end procedure

```

4.3 Lightweight routing on the hot path (C2)

This mechanism addresses C2 by shortening the access chain from a logical state operation to a physical slot probe. While C1 focuses on the memory layout *within* a table, C2 focuses on reducing dependent pointer chasing *before* probing begins. Fewer layers and indirections reduce the serial stall component on the critical path and improve the constant factors in T_{cpu} (Eq. 3).

A long access chain (map lookups, wrapper objects, nested containers) creates a sequence of dependent memory accesses where each next pointer is only known after the previous load returns. This serialization reduces memory-level parallelism and increases sensitivity to LLC/DRAM latency. By routing in a small number of predictable steps, we reduce the number of dependent loads required before reaching the metadata probe.

Concretely, state is partitioned by key group, and further segmented by namespace for windows and operator sub-state. Each access selects the key-group shard, then the namespace-local fragment, and finally probes a compact sub-table. For effectively un-namespaced state, the fast path avoids an extra mapping layer and directly indexes a per-key-group table; for general namespaces, each key group maintains a small namespace map whose values are independent sub-tables.

This routing also specializes the common case. In many operators, a large fraction of accesses fall into a single effective namespace (or a small fixed set). A specialized fast path for this case avoids a general-purpose namespace map, reducing both pointer chasing and cache churn in routing metadata. In terms of Eq. (3), this reduces the overhead inside T_{cpu} and can also lower miss intensity by touching fewer routing structures per operation.

Algorithm 2 Insert/update with tombstones and maintenance triggers.

```

1: procedure PUT( $T, h, k, v$ )
2:    $g \leftarrow \text{GROUPINDEX}(h)$ 
3:    $f \leftarrow \text{FINGERPRINT}(h)$ 
4:    $firstAvail \leftarrow \perp$   $\triangleright$  best EMPTY/DELETED slot observed so
   far
5:   loop
6:      $ctrl \leftarrow \text{LOADCTRLWORD}(T, g)$ 
7:      $mask \leftarrow \text{MATCHFINGERPRINT}(ctrl, f)$ 
8:     while  $mask \neq 0$  do
9:        $i \leftarrow \text{NEXTLANE}(mask)$ 
10:       $s \leftarrow \text{GROUPBASE}(g) + i$ 
11:       $a \leftarrow \text{SLOTKEYASSOC}(T, s)$ 
12:      if  $\text{KEYEQUALS}(a, k)$  then
13:         $\text{WRITEVALUE}(T, s, v)$ 
14:        return UPDATED
15:      end if
16:       $mask \leftarrow \text{CLEARLANE}(mask, i)$ 
17:    end while
18:    if  $firstAvail = \perp$  then
19:       $firstAvail \leftarrow \text{FIRSTEMPTYORDELETED}(ctrl, g)$ 
20:    end if
21:    if  $\text{HASEMPTY}(ctrl)$  then
22:      break
23:    end if
24:     $g \leftarrow \text{NEXTGROUP}(g)$ 
25:  end loop
26:   $s \leftarrow firstAvail$ 
27:   $\text{WRITEKEYASSOC}(T, s, k)$   $\triangleright$  optionally store compact
   fixed-width representation when available
28:   $\text{WRITEVALUE}(T, s, v)$ 
29:   $\text{SETCTRL}(T, s, f)$ 
30:  if  $\text{NEEDMAINTENANCE}(T)$  then
31:     $\text{REQUESTREHASHORGROW}(T)$ 
32:  end if
33:  return INSERTED
34: end procedure

```

Placeholder. Replace this box with a hierarchy diagram. Suggested content: an array of key-group shards; within each shard either (i) direct table for the common un-namespaced case, or (ii) a small namespace map to independent sub-tables; annotate that each namespace has an isolated working set.

Figure 6: Hierarchical state organization that aligns physical layout with key-group and namespace execution context.

Figure 6 visualizes the hierarchical organization (KeyGroup $\rightarrow$ Namespace $\rightarrow$ sub-tables) and highlights the fast path for effectively un-namespaced state.

Algorithm 3 Maintenance: rehash/grow via physical-layout rebuild.

```

1: procedure REBUILD( $T, mode$ )     $\triangleright mode \in \{\text{REHASH, GROW}\}$ 
2:   if  $mode = \text{REHASH}$  then
3:      $cap' \leftarrow cap(T)$ 
4:   else
5:      $cap' \leftarrow \text{NEXTCAPACITY}(cap(T))$ 
6:   end if
7:    $T' \leftarrow \text{ALLOCATETABLE}(cap')$   $\triangleright$  control bytes initialized to
  EMPTY
8:   for all slots  $s$  in  $T$  do
9:      $c \leftarrow \text{SLOTCTRL}(T, s)$ 
10:    if  $\text{IsFULL}(c)$  then
11:       $(k, v, h) \leftarrow \text{READLOGICALENTRY}(T, s)$ 
12:       $\text{PUT}(T', h, k, v)$   $\triangleright$  re-insert using metadata-guided
        probing
13:    end if
14:  end for
15:   $\text{REPLACE}(T, T')$   $\triangleright$  swap references; reclaim old table
16: end procedure

```

4.4 Namespace-partitioned organization for locality isolation (C3)

This mechanism addresses C3 by aligning physical partitioning with streaming execution context so that hot working sets stay small and predictable. Intuitively, isolating state by key group and namespace reduces cache interference: frequently accessed namespaces do not have to compete with cold namespaces inside the same large table.

We structure state hierarchically as $\text{KeyGroup} \rightarrow \text{Namespace} \rightarrow \text{sub-table}$. At the top level, an operator instance holds an array of key-group shards. Within each key group, state is further separated by namespace, and each namespace maps to an independent compact sub-table. This organization yields two locality benefits: (i) it bounds the working set that any single hot context must touch, and (ii) it reduces the probability that unrelated namespaces evict each other's metadata and payload from cache.

Namespaces can become cold or empty as windows close and timers fire. When a namespace table becomes empty, we remove it from the namespace map so that cold namespaces do not accumulate as routing overhead or consume cache capacity. This keeps the per-key-group namespace metadata compact and improves locality under long-running jobs with many transient namespaces.

Timer firing and window maintenance naturally concentrate accesses into a small set of active namespaces for short periods. By keeping those namespaces in separate sub-tables, bursts are less likely to churn through unrelated cold state, reducing conflict/capacity misses and stabilizing tail latency.

Namespace isolation reduces both Miss_{LLC} and $\text{Miss}_{\text{DRAM}}$ in Eq. (3) by shrinking the effective working set per hot context and lowering cache interference under mixed workloads.

5 Implementation Details

This section highlights the minimal implementation choices needed to reproduce the design in §4 in a Flink-compatible state backend.

We keep it intentionally brief: the design rationale is fully captured in §4.

5.1 Table representation and hash decomposition

Each namespace-local sub-table is an open-addressed structure with (i) a compact control-byte array (one byte per slot) and (ii) a dense slot array that stores the key association adjacent to the value (slot-centric layout). Control bytes encode $\text{EMPTY/DELETED/FULL}$ and store a short fingerprint; probing scans control metadata in 8-slot groups and touches payload only for candidate lanes (Alg. 1).

We compute a 32-bit mixed hash and split it into high bits H_1 (group selection) and low bits H_2 (fingerprint stored in control bytes). To keep hits localized, each slot stores key association and value contiguously; when a compact fixed-width key representation is available, we inline it to reduce extra dereferences during validation.

5.2 Routing and namespace fast path

We implement the $\text{KeyGroup} \rightarrow \text{Namespace} \rightarrow \text{sub-table}$ organization described in §4. An operator instance holds an array of key-group shards; within each shard, state is either routed through a fast path for the effectively un-namespaced case (e.g., VOIDNAMESPACE) by directly indexing a per-key-group table, or through a small namespace map whose values are independent sub-tables.

To keep routing overhead bounded under long-running jobs, we remove a namespace entry when its sub-table becomes empty after deletions. This avoids retaining cold namespaces as persistent routing metadata and helps preserve locality isolation.

5.3 Maintenance: rehash and grow

We trigger maintenance based on probe-efficiency signals rather than on fixed operation counts. In particular, we track occupancy and tombstone ratio; high tombstone density motivates rehashing, while high occupancy motivates growth. Both operations rebuild the physical layout by allocating a fresh table, reinserting logical entries, and reconstructing control metadata (Alg. 3).

5.4 Flink integration and access path

We implement the design by plugging into Flink's standard keyed-state abstractions. Our backend provides a keyed state backend that owns the per-operator state store and is responsible for routing each state access to the correct key group and namespace.

On each record, Flink establishes the current key context (and thus the current key group). A state access then resolves the namespace from the state descriptor/serializer context and performs routing in the same order as §4: (i) index the key-group shard, (ii) select the namespace-local sub-table (either via the fast path for VOIDNAMESPACE or via the per-key-group namespace map), and (iii) execute the metadata-guided probe for lookup/update (Alg. 1 and Alg. 2). This organization makes the implementation correspond directly to the three design mechanisms: compact metadata-driven probing and layout (C1), lightweight routing on the hot path (C2), and namespace-partitioned state organization for locality isolation (C3).

Table 1: Workloads used in evaluation and the primary behaviors they stress.

Workload	Level	What it stresses
flink-state-benchmark microbenchmarks	Micro	Isolated state lookups/updates; sweeps over working-set size, skew, read/write mix, and churn
benchset end-to-end benchmarks	E2E	Full pipelines; sustained updates; windowed state and timer-driven bursts; interaction with scheduling and GC

6 Experiments

6.1 Setup

We run experiments on a dedicated multi-core Linux server to avoid interference. We report CPU model, core count, cache hierarchy, DRAM capacity, and NUMA configuration (**fill in final numbers**). We use Apache Flink **1.20.0** with a fixed JDK version (**fill in**) and keep Flink job logic, serializers, and parallelism identical across runs; the only change between runs is the state backend.

Unless otherwise stated, we compare: (i) the **proposed backend** and (ii) **HashMapStateBackend** (the standard in-memory heap backend; often referred to as **HeapStateBackend** in earlier Flink versions). We optionally include **RocksDBStateBackend** to contextualize in-memory vs. embedded-KV trade-offs; when included we report its configuration and interpret it as a different point in the design space.

6.2 Methodology

Protocol. For each configuration point, we run to steady state, discard warm-up, and report the median across repeated trials. Unless otherwise stated, we use a fixed-duration warm-up (e.g., 2 minutes) followed by a fixed-duration measurement window (e.g., 5 minutes), repeat 5 times with identical inputs and random seeds, and report median with tail percentiles. We report tail percentiles because timer bursts and maintenance can amplify variance.

Workloads. We use two complementary workload families: (i) flink-state-benchmark microbenchmarks to isolate state-access behavior under systematic sweeps, and (ii) benchset end-to-end pipelines to validate application-level impact.

Metrics. We report throughput and latency (including tail percentiles). To connect performance to memory-system effects, we collect cache-miss intensity and a memory-bound breakdown from hardware counters. For internal behavior, we report maintenance frequency (rehash/grow), memory footprint, and (when enabled) checkpoint overhead.

Parameter sweeps. Across RQs we systematically sweep: working-set size (from cache-resident to DRAM-resident), operation mix (read-heavy to write-heavy), key distribution (uniform to Zipf-skewed; e.g., multiple Zipf parameters), namespace cardinality (e.g., 1, 10^3 , 10^5 namespaces), and churn (insert/update/delete mixes; e.g., 0%/20%/50% deletes). We keep operator logic and Flink configuration fixed while varying exactly one axis at a time.

Experimental design (research questions). The core of our methodology is a question-driven experiment matrix. Each question below states what we want to establish and the experiments we design to establish it.

- **RQ1 (End-to-end impact).** *What is the end-to-end benefit of the proposed backend for realistic stateful streaming pipelines?* Design: run the benchset end-to-end suite (including at least one windowed and one timer-heavy pipeline) with identical Flink configurations across backends; report sustained throughput and latency percentiles (p50/p95/p99/p99.9). For one timer/window-heavy pipeline, additionally report a time series of throughput and high-percentile latency to expose burst-induced tail effects (e.g., timer firing bursts and maintenance spikes).
- **RQ2 (Backend-level comparison).** *How does the proposed backend compare under controlled state-access patterns, and where do improvements appear as the working set crosses cache boundaries?* Design: use flink-state-benchmark microbenchmarks to sweep working-set size across cache boundaries (L1/L2/L3 and DRAM-resident), and vary operation mixes (e.g., 100% read, 50/50 read-update, update-heavy) and key/namespace distributions (uniform vs. Zipf-skewed). We report both throughput and tail latency, and mark the approximate cache-capacity transition points based on the working-set footprint.
- **RQ3 (Internal mechanisms).** *Which design mechanisms contribute most: C1 (metadata-guided probing/layout), C2 (lightweight routing), or C3 (namespace partitioning)?* Design: perform ablations that disable one mechanism at a time while keeping the rest identical (C1-only removed, C2-only removed, C3-only removed). We run the same microbenchmarks as in RQ2 and at least one representative end-to-end pipeline from RQ1, and report (i) normalized throughput/latency and (ii) hardware-counter deltas to connect each mechanism to cache behavior.
- **RQ4 (Workload sensitivity).** *How robust is the design across locality regimes and churn patterns?* Design: sweep (i) key skew (uniform vs. Zipf with multiple skew parameters), (ii) namespace cardinality (single namespace vs. many namespaces) and hot-cold mixes, (iii) churn (insert/update/delete mixes that create tombstones), and (iv) burst intensity (timer/window configurations that vary firing density). For each sweep we report throughput, tail latency, and maintenance counters (rehash/grow frequency and time) to quantify robustness under adversarial locality and mutation.
- **RQ5 (Micro-architectural evidence).** *Do hardware counters support the locality hypothesis, and do they correlate with throughput/latency improvements?* Design: collect hardware counters for representative micro and end-to-end runs; report cache-miss intensity (e.g., last-level-cache misses per record and per instruction) and a top-down style memory-bound breakdown. We use Linux `perf` for miss-intensity and cycle/instruction accounting, and Intel VTune to attribute stalls into front-end / back-end / memory-bound components.

Table 2: Metrics reported and how they are collected.

Metric	Definition	Collection
Throughput	Records processed per second under steady-state load	Benchmark harness reports; validated by run-time counters
Latency	Per-record processing latency, including tail percentiles	Benchmark harness timestamps and percentile summaries
Cache miss rate	Miss intensity of relevant cache levels during execution	Hardware performance counters (Linux perf/perf_event)
Memory-bound breakdown	Fraction of cycles attributable to memory stalls vs. retiring work	Intel VTune top-down analysis / stall attribution
Maintenance overhead	Rehash/grow frequency and time; impact on tail latency	Backend-instrumented counters + benchmark timestamps
Memory footprint	Bytes per logical entry (including metadata)	Heap/off-heap accounting; JVM tooling (fill in)
Checkpoint overhead	Checkpoint time, size, and restore time (if enabled)	Flink checkpoint metrics and logs

Placeholder (RQ1). End-to-end throughput and tail latency. Suggested content: per-workload throughput; latency percentiles; and a timer/window-heavy case study.

Figure 7: RQ1: End-to-end impact on throughput and tail latency.

- **RQ6 (Overheads and trade-offs).** *What are the costs: memory footprint, maintenance overhead, and checkpoint/recovery impact? Design:* measure bytes per logical entry (payload vs. metadata), GC activity, rehash/grow frequency and time (and its contribution to tail outliers), and checkpoint time/size (and restore time if evaluated). We report both steady-state overheads (bytes/entry, CPU overhead of maintenance) and operational overheads (checkpoint/recovery) to make the trade-offs explicit.

6.3 Results

This subsection reports the measured results for RQ1–RQ6. Figures are grouped by end-to-end performance, microbenchmark characterization, ablations, sensitivity, hardware-counter evidence, and overheads.

6.4 Discussion

The evaluation is designed to test the hypothesis that state performance in stream processing is primarily limited by memory-system behavior rather than compute, and that reducing cache misses is an effective lever to improve both throughput and tail latency. The strongest benefits occur under windowed and timer-heavy workloads, where frequent updates and maintenance bursts repeatedly exercise the same key-group and namespace-local working sets. By shortening the access path and making probing more cache-guided, the design reduces stall-dominated execution and stabilizes performance under mixed access patterns. When locality is already high or the working set fits comfortably in cache, improvements remain but are naturally smaller, because the baseline pays fewer DRAM-level misses to begin with. The design also introduces modest metadata overhead (control bytes, stored hashes) and occasionally performs maintenance (e.g., rehashing to clear tombstones or growing the table) to preserve probe efficiency; these costs are amortized by the steady-state reduction in cache-line touches on the critical per-record path.

An important implication is that steady-state locality optimizations are complementary to checkpoint/recovery optimizations. Faster checkpointing, incremental snapshotting, and parallel recovery reduce the time spent off the critical path during failures and recovery, while locality-driven probing reduces the per-record cost during normal execution. In practice, both costs matter: large-state jobs can be bottlenecked by steady-state access at high update rates and also constrained by recovery time objectives. A cache-friendly physical layout can therefore improve day-to-day throughput and tail latency while remaining compatible with system-level advances in fault tolerance.

7 Related Work

7.1 State management in stream processing

Large-scale stream processors have established the core semantics and mechanisms for fault-tolerant stateful processing, including keyed state, windowing/timers, and checkpoint/recovery protocols [1–5, 9, 10]. A recurring theme in these systems is that correctness and recovery are achieved through well-defined state abstractions and consistent snapshots; the remaining performance bottleneck increasingly shifts to the steady-state cost of fine-grained per-record state access at high update rates. Recent work has also explored elasticity and online state movement to cope with workload shifts, e.g., latency-conscious state migration in distributed dataflows [11]. Complementary to these directions, our paper targets the micro-architectural efficiency of the hot path itself: we preserve the state abstraction and snapshot contract, but redesign in-memory layout and probing so that lookups/updates touch fewer cache lines.

Recent work also revisits the performance costs of checkpointing and recovery, which can dominate for large state and frequent snapshots. Techniques include incremental checkpointing based on data-structure design [23], analyses of how storage-layer behaviors such as log compaction interact with incremental checkpointing [24], and parallel recovery mechanisms that reduce restart time by recovering state fragments concurrently [25]. Complementary studies bridge theory and practice for checkpoint-interval tuning

Placeholder (RQ2). Microbenchmark sweeps. Suggested content: throughput and latency vs. working-set size; separate panels for read-heavy vs. write-heavy mixes; include uniform vs. skewed distributions.

Figure 8: RQ2: Backend-level comparison under controlled state-access patterns.

Placeholder (RQ3). Ablation results. Suggested content: normalized throughput/latency and cache-miss intensity for each ablation.

Figure 9: RQ3: Ablation study isolating contributions of major design mechanisms.

Placeholder (RQ4). Robustness and sensitivity. Suggested content: performance and tail-latency trends under skew, namespace cardinality, churn, and burst intensity.

Figure 10: RQ4: Workload sensitivity across locality and churn regimes.

and end-to-end fault-tolerance overheads in distributed streaming systems [28]. While these works focus on snapshot/recovery overheads and their scaling behavior, we focus on the steady-state per-record access path and its cache efficiency; the two directions are compatible and can be composed.

At the methodology level, a growing body of work emphasizes the need for systematic evaluation of streaming state stores and their operational trade-offs [13]. Our evaluation follows this spirit, but focuses specifically on connecting cache behavior (miss intensity, stall time) to state-store design choices.

7.2 Distributed state backends and storage architectures

Beyond purely in-process state, many deployments rely on embedded or external storage components (e.g., LSM-based engines) to trade CPU/cache efficiency for capacity and durability. Recent research revisits this design space with modern hardware: SSD-backed state stores reduce memory pressure but introduce I/O-aware access paths [16], while RDMA-enabled designs push state access toward disaggregated or remote-memory architectures to reduce CPU overheads and improve scaling [15]. Other systems propose specialized state stores for stream engines, e.g., semantic-aware organization for large-scale state management [17].

At larger scales, distributed state management introduces additional challenges around partitioning, placement, and metadata management. Rhino studies efficient management of very large distributed state for stream processing engines [14]. Amoeba aligns stream operators with externally managed state to support elasticity and operational flexibility [19]. Related work also explores general strategies for scaling out stateful operators [18]. Our focus is orthogonal: we assume the standard per-task state interface

and snapshot semantics, and improve the cache behavior of the in-memory access path that remains performance-critical even when the system adopts larger-scale state backends.

7.3 Cache-friendly hash table and indexing techniques

There is a long line of work on cache- and locality-aware hashing and indexing, including compact open addressing and collision-resolution schemes such as cuckoo hashing and hopscotch hashing [6, 7], as well as cache-conscious techniques for reducing random accesses [8]. Micro-architectural analyses and optimizations have shown how probe structure, branch behavior, and memory access patterns dominate hash-table throughput on modern CPUs [21]. More recently, SIMD-aware hash-table designs leverage vector instructions to accelerate matching and reduce control-flow divergence [22], and new hash-table designs explicitly optimize cache-line touches and write amplification under contemporary memory hierarchies [20].

Our design draws on these principles (metadata-first filtering, word-parallel matching, and cache-resident control information) but adapts them to streaming state: the hot path is dominated by repeated lookups/updates under continuous mutation, and the state store must preserve snapshot/restore semantics and namespace organization. Consequently, we emphasize a shortened access path and a physical layout aligned with key-group and namespace locality, rather than optimizing a standalone dictionary in isolation.

8 Conclusion

This paper argues that the limiting factor for large-state stream processing is increasingly the memory behavior of state access rather than the compute cost of operator logic. By making locality a first-class design goal, we show how compact indexing with parallel matching, inlined layouts, and a shortened access path can reduce cache-miss intensity while preserving state semantics and snapshot/restore compatibility. Organizing state by key-group and namespace further aligns physical layout with the execution context of windows and timers, helping isolate working sets and stabilize tail latency under mixed workloads. Our evaluation across isolated microbenchmarks and end-to-end pipelines demonstrates consistent improvements in throughput and latency trends, and provides evidence of a shift away from memory-bound execution. These results suggest that cache-aware state storage is a practical lever for scaling stateful operators in distributed and parallel streaming systems. Looking forward, we believe the same locality-first principles can be extended to multi-tier state hierarchies and adaptive partitioning policies that react to workload shifts while maintaining operational compatibility.

Placeholder (RQ5). Memory-system indicators. Suggested content: cache-miss intensity and a memory-bound breakdown aligned with RQ1/RQ2 runs.

Figure 11: RQ5: Hardware-counter evidence for reduced memory-system pressure.

Placeholder (RQ6). Overheads and trade-offs. Suggested content: bytes/entry; maintenance frequency; checkpoint time/size; and their impact on tail latency.

Figure 12: RQ6: Overheads and trade-offs (footprint, maintenance, checkpointing).

Acknowledgments

We thank the anonymous reviewers for their valuable feedback.

References

- [1] Tyler Akidau, Alex Balikov, Kaya Bekiroglu, Slava Chernyak, Josh Haberman, Reuven Lax, Sam McVeety, Daniel Mills, Paul Nordstrom, and Sam Whittle. 2013. *MillWheel: Fault-Tolerant Stream Processing at Internet Scale*. Proceedings of the VLDB Endowment (PVLDB) 6, 11 (2013), 1033–1044. <https://doi.org/10.14778/2536222.2536229>
- [2] Tyler Akidau, Robert Bradshaw, Craig Chambers, Slava Chernyak, Rafael Fernández-Moctezuma, Reuven Lax, Sam McVeety, Daniel Mills, Frances Perry, Eric Schmidt, and Sam Whittle. 2015. *The Dataflow Model: A Practical Approach to Balancing Correctness, Latency, and Cost in Massive-Scale, Unbounded, Out-of-Order Data Processing*. Proceedings of the VLDB Endowment (PVLDB) 8, 12 (2015), 1792–1803. <https://doi.org/10.14778/2824032.2824076>
- [3] Paris Carbone, Stephan Ewen, Gyula Fóra, Seif Haridi, Stefan Richter, and Kostas Tzoumas. 2017. *State Management in Apache Flink®: Consistent Stateful Distributed Stream Processing*. Proceedings of the VLDB Endowment (PVLDB) 10, 12 (2017), 1718–1729. <https://doi.org/10.14778/3137765.3137777>
- [4] Michael Armbrust, Tathagata Das, Joseph Torres, Burak Yavuz, Shixiong Zhu, Reynold Xin, Ali Ghodsi, Ion Stoica, Matei Zaharia. 2018. *Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark*. In Proceedings of the 2018 International Conference on Management of Data (SIGMOD). ACM, 601–613. <https://doi.org/10.1145/3183713.3190664>
- [5] Derek Gordon Murray, Frank McSherry, Rebecca Isaacs, Michael Isard, Paul Barham, and Martin Abadi. 2013. *Naiad: A Timely Dataflow System*. In Proceedings of the 24th ACM Symposium on Operating Systems Principles (SOSP). ACM, 439–455. <https://doi.org/10.1145/2517349.2522738>
- [6] Rasmus Pagh and Flemming Friche Rodler. 2004. *Cuckoo Hashing*. Journal of Algorithms 51, 2 (2004), 122–144. <https://doi.org/10.1016/j.jalgor.2003.12.002>
- [7] Maurice Herlihy, Nir Shavit, and Moran Tzafrir. 2008. *Hopscotch Hashing*. In Distributed Computing (DISC), Lecture Notes in Computer Science 5218. Springer, 350–364. https://doi.org/10.1007/978-3-540-87779-0_24
- [8] Nikolas Askitis and Justin Zobel. 2005. *Cache-Conscious Collision Resolution in String Hash Tables*. In String Processing and Information Retrieval (SPIRE), Lecture Notes in Computer Science 3772. Springer, 91–102. https://doi.org/10.1007/11575832_11
- [9] Badrish Chandramouli, Jonathan Goldstein, Mike Barnett, and Robert DeLine. 2014. *Trill: A High-Performance Incremental Query Processor for Diverse Analytics*. Proceedings of the VLDB Endowment (PVLDB) (2014). <https://doi.org/10.14778/2735496.2735503>
- [10] John Meehan, Nesime Tatbul, Stan Zdonik, and Cansu Aslantas. 2015. *S-Store: Streaming Meets Transaction Processing*. Proceedings of the VLDB Endowment (PVLDB) (2015). <https://doi.org/10.14778/2831360.2831367>
- [11] Moritz Hoffmann, Andrea Lattuada, and Frank McSherry. 2019. *Megaphone: Latency-Conscious State Migration for Distributed Streaming Dataflows*. Proceedings of the VLDB Endowment (PVLDB) (2019). <https://doi.org/10.14778/3329772.3329777>
- [12] Yanif Ahmad, Oliver Kennedy, Christoph Koch, and Milos Nikolic. 2012. *DBToaster: Higher-Order Delta Processing for Dynamic, Frequently Fresh Views*. Proceedings of the VLDB Endowment (PVLDB) (2012). <https://doi.org/10.14778/2336664.2336670>
- [13] Esmail Asyabi, Yuanli Wang, John Liagouris, Vasiliki Kalavri, and Azer Bestavros. 2022. *A new benchmark harness for systematic and robust evaluation of streaming state stores*. In Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys '22). ACM, 559–574. <https://doi.org/10.1145/3492321.3519592>
- [14] Bonaventura Del Monte, Steffen Zeuch, Tilmann Rabl, and Volker Markl. 2020. *Rhino: Efficient Management of Very Large Distributed State for Stream Processing Engines*. In Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD/PODS '20). ACM, 2471–2486. <https://doi.org/10.1145/3318464.3389723>
- [15] Bonaventura Del Monte, Steffen Zeuch, Tilmann Rabl, and Volker Markl. 2022. *Rethinking Stateful Stream Processing with RDMA*. In Proceedings of the 2022 International Conference on Management of Data (SIGMOD/PODS '22). ACM, 1078–1092. <https://doi.org/10.1145/3514221.3517826>
- [16] Gyewon Lee, Jeongyoon Eo, Jangho Seo, Taegeon Um, and Byung-Gon Chun. 2018. *High-Performance Stateful Stream Processing on Solid-State Drives*. In Proceedings of the 9th Asia-Pacific Workshop on Systems (APSys '18). ACM, 1–7. <https://doi.org/10.1145/3265723.3265739>
- [17] Gyewon Lee, Jaewoo Maeng, Jinsol Park, Jangho Seo, Haeyoon Cho, Youngseok Yang, Taegeon Um, Jongsung Lee, Jae W. Lee, and Byung-Gon Chun. 2023. *FlowKV: A Semantic-Aware Store for Large-Scale State Management of Stream Processing Engines*. In Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys '23). ACM, 768–783. <https://doi.org/10.1145/3552326.3567493>
- [18] Muhammad Mudassar, Yanlong Zhai, and Lejian Liao. 2019. *Efficient State Management for Scaling Out Stateful Operators in Stream Processing Systems*. Big Data 7, 3 (2019), 192–206. <https://doi.org/10.1089/big.2018.0093>
- [19] Antonis Papaioannou and Kostas Magoutis. 2021. *Amoeba: aligning stream processing operators with externally-managed state*. In Proceedings of the 14th IEEE/ACM International Conference on Utility and Cloud Computing (UCC '21). ACM, 1–10. <https://doi.org/10.1145/3468737.3494096>
- [20] Prashant Pandey, Michael A. Bender, Alex Conway, Martin Farach-Colton, William Kuszmaul, Guido Tagliavini, and Rob Johnson. 2023. *IcebergHT: High Performance Hash Tables Through Stability and Low Associativity*. Proceedings of the ACM on Management of Data 1, 1 (2023), 1–26. <https://doi.org/10.1145/3588727>
- [21] Kenneth A. Ross. 2007. *Efficient Hash Probes on Modern Processors*. In 2007 IEEE 23rd International Conference on Data Engineering (ICDE). IEEE, 1297–1301. <https://doi.org/10.1109/ICDE.2007.368997>
- [22] Dipti Shankar, Xiaoyi Lu, and Dhabaleswar K. Panda. 2019. *SIMD-Aware Hash Table Designs on Emerging CPU Architectures*. In 2019 IEEE International Symposium on Workload Characterization (IISWC). IEEE. <https://doi.org/10.1109/IISWC47752.2019.9042069>
- [23] Chia-Yu Lin, Li-Chun Wang, and Shu-Ping Chang. 2022. *Incremental Checkpointing for Fault-Tolerant Stream Processing Systems: A Data Structure Approach*. IEEE Transactions on Emerging Topics in Computing 10, 1 (2022), 124–136. <https://doi.org/10.1109/TETC.2020.2986487>
- [24] Aris Chronarakis, Antonis Papaioannou, and Kostas Magoutis. 2019. *On the Impact of log Compaction on Incrementally Checkpointing Stateful Stream-Processing Operators*. In 2019 38th International Symposium on Reliable Distributed Systems Workshops (SRDSW). IEEE, 1–6. <https://doi.org/10.1109/SRDSW49218.2019.00008>
- [25] Pinchao Liu, Hailu Xu, Dilma Da Silva, Qingyang Wang, Sarker Tanzir Ahmed, and Liting Hu. 2020. *FP4S: Fragment-based Parallel State Recovery for Stateful Stream Applications*. In 2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS). IEEE, 1102–1111. <https://doi.org/10.1109/IPDPS47924.2020.00116>
- [26] Shuhao Zhang, Yingjun Wu, Feng Zhang, and Bingsheng He. 2020. *Towards Concurrent Stateful Stream Processing on Multicore Processors*. In 2020 IEEE 36th International Conference on Data Engineering (ICDE). IEEE, 1537–1548. <https://doi.org/10.1109/ICDE48307.2020.00136>
- [27] K. Sornalakshmi and G. Vadivu. 2023. *An empirical analysis of stateful operator migration for online scheduling in distributed stream processing systems*. Microprocessors and Microsystems 98 (2023), 104811. <https://doi.org/10.1016/j.micpro.2023.104811>
- [28] Sachini Jayasekara, Shanika Karunasekera, and Aaron Harwood. 2022. *Optimizing checkpoint-based fault-tolerance in distributed stream processing systems: Theory to practice*. Software: Practice and Experience 52, 1 (2022), 296–315. <https://doi.org/10.1002/spe.3021>